

NOTAZIONE ASINTOTICA

Definizioni Formali

O-grande (limite superiore)

$$f(n) = O(g(n)) \Leftrightarrow \exists c > 0, n_0 \geq 0: \forall n \geq n_0, f(n) \leq c \cdot g(n)$$

Ω -grande (limite inferiore)

$$f(n) = \Omega(g(n)) \Leftrightarrow \exists c > 0, n_0 \geq 0: \forall n \geq n_0, f(n) \geq c \cdot g(n)$$

Θ -grande (limite stretto)

$$f(n) = \Theta(g(n)) \Leftrightarrow f(n) = O(g(n)) \wedge f(n) = \Omega(g(n))$$

Proprietà Fondamentali

- $f(n) = O(g(n)) \Leftrightarrow g(n) = \Omega(f(n))$
- $\Theta(g(n)) = O(g(n)) \cap \Omega(g(n))$
- $f(n) = \Theta(f(n))$ (riflessività)
- $f(n) = O(g(n)) \wedge g(n) = O(h(n)) \Rightarrow f(n) = O(h(n))$ (transitività)

MASTER THEOREM

Per ricorrenze nella forma: $T(n) = aT(n/b) + f(n)$

dove $a \geq 1$, $b > 1$, $f(n)$ asintoticamente positiva.

Parametri

- a = numero di sottoproblemi
- b = fattore di riduzione della taglia
- $f(n)$ = costo combinazione soluzioni
- $n^{\log_b a}$ = funzione di confronto

Tre Casi

CASO 1: $f(n) = O(n^{\log_b a - \epsilon})$ per qualche $\epsilon > 0 \rightarrow T(n) = \Theta(n^{\log_b a})$

Domina il costo dei sottoproblemi

CASO 2: $f(n) = \Theta(n^{\log_b a}) \rightarrow T(n) = \Theta(n^{\log_b a} \cdot \log n)$ *Bilanciamento tra sottoproblemi e combinazione*

CASO 3: $f(n) = \Omega(n^{\log_b a + \epsilon})$ per qualche $\epsilon > 0$ E $a \cdot f(n/b) \leq k \cdot f(n)$ per qualche $k < 1$ e n sufficientemente grande (regolarità) $\rightarrow T(n) = \Theta(f(n))$ *Domina il costo di combinazione*

Esempi Applicati

$T(n) = 2T(n/2) + n \log n$	$\rightarrow a=2, b=2, \log_2 2=1, f(n)=\Theta(n^1) \rightarrow \text{CASO 2} \rightarrow \Theta(n \log n)$
$T(n) = 4T(n/2) + n$	$\rightarrow a=4, b=2, \log_2 4=2, f(n)=O(n^1) \rightarrow \text{CASO 1} \rightarrow \Theta(n^2)$
$T(n) = 2T(n/2) + n^2$	$\rightarrow a=2, b=2, \log_2 2=1, f(n)=\Omega(n^2) \rightarrow \text{CASO 3} \rightarrow \Theta(n^2)$
$T(n) = T(n/3) + T(2n/3) + n$	$\rightarrow \text{Master Theorem NON applicabile (coefficienti diversi)}$
$T(n) = 6T(n/3) + n^2$	$\rightarrow a=6, b=3, \log_3 6 \approx 1.63, f(n)=\Theta(n^2) \rightarrow \text{CASO 3} \rightarrow \Theta(n^2)$

METODO DI SOSTITUZIONE

Per ricorrenze non risolvibili con Master Theorem.

Procedura

1. **Ipotizza** la soluzione $T(n) = O(f(n))$ o $T(n) = \Theta(f(n))$
2. **Dimostra per induzione** matematica:
 - **Caso base:** verifica per piccoli valori di n
 - **Passo induttivo:** assumi vero per $k < n$, dimostra per n
3. **Trova costanti** c, n_0 che soddisfano la definizione

Esempio: $T(n) = T(n/3) + T(2n/3) + \Theta(n)$

Ipotesi: $T(n) = O(n \log n)$

Dimostrazione: Assumiamo $T(k) \leq c \cdot k \cdot \log k$ per ogni $k < n$.

$$\begin{aligned} T(n) &= T(n/3) + T(2n/3) + \Theta(n) \\ &\leq c(n/3)\log(n/3) + c(2n/3)\log(2n/3) + dn \\ &= (cn/3)(\log n - \log 3) + (2cn/3)(\log n - \log 3) + dn \\ &= cn \cdot \log n - cn \cdot \log 3 + dn \\ &\leq cn \cdot \log n \quad \text{se } c \geq d/\log 3 \end{aligned}$$

Quindi $T(n) = O(n \log n)$ ✓



COMPLESSITÀ ALGORITMI COMUNI

Ordinamento

Algoritmo	Caso peggiore	Caso medio	Caso migliore	Spazio
InsertionSort	$\Theta(n^2)$	$\Theta(n^2)$	$\Theta(n)$	$O(1)$
MergeSort	$\Theta(n \log n)$	$\Theta(n \log n)$	$\Theta(n \log n)$	$O(n)$
QuickSort	$\Theta(n^2)$	$\Theta(n \log n)$	$\Theta(n \log n)$	$O(\log n)$
HeapSort	$\Theta(n \log n)$	$\Theta(n \log n)$	$\Theta(n \log n)$	$O(1)$
CountingSort	$\Theta(n+k)$	$\Theta(n+k)$	$\Theta(n+k)$	$O(n+k)$
RadixSort	$\Theta(d(n+k))$	$\Theta(d(n+k))$	$\Theta(d(n+k))$	$O(n+k)$

Strutture Dati - Operazioni

Struttura	Ricerca	Inserimento	Cancellazione	Spazio
Array ordinato	$O(\log n)$	$O(n)$	$O(n)$	$O(n)$
Hash Table	$O(1)$ atteso	$O(1)$ atteso	$O(1)$ atteso	$O(n)$
Heap	$O(n)$	$O(\log n)$	$O(\log n)$	$O(n)$
ABR	$O(h)$	$O(h)$	$O(h)$	$O(n)$
RB-Tree	$O(\log n)$	$O(\log n)$	$O(\log n)$	$O(n)$

Programmazione Dinamica Classici

Problema	Complessità Tempo	Complessità Spazio
LCS	$\Theta(mn)$	$O(mn)$ o $O(\min(m,n))$
LIS	$\Theta(n^2)$	$O(n)$
Shortest Palindrome Completion	$\Theta(n^2)$	$O(n^2)$
Coin Change	$\Theta(nK)$	$O(n)$



LIMITI INFERIORI

Ordinamento Basato su Confronti

Limite inferiore: $\Omega(n \log n)$

Dimostrazione: Albero delle decisioni con $n!$ foglie

$$\text{altezza} \geq \log_2(n!) = \Theta(n \log n)$$

Scambi di Elementi Contigui

Ordinare array con solo scambi di elementi adiacenti: $\Omega(n^2)$

Dimostrazione: Numero di inversioni nel caso peggiore = $n(n-1)/2 = \Theta(n^2)$



SOMMATORIE UTILI

$$\sum_{i=1 \text{ to } n} i = n(n+1)/2 = \Theta(n^2)$$

$$\sum_{i=1 \text{ to } n} i^2 = n(n+1)(2n+1)/6 = \Theta(n^3)$$

$$\sum_{i=0 \text{ to } n} 2^i = 2^{n+1} - 1 = \Theta(2^n)$$

$$\sum_{i=1 \text{ to } n} 1/i = \ln(n) + \gamma = \Theta(\log n) \quad (\text{serie armonica})$$

$$\sum_{i=0 \text{ to } \log n} n/2^i = 2n - n/(2^{\log n}) = \Theta(n) \quad (\text{tecnica telescopica})$$



TRUCCHI PER L'ESAME

Riconoscere la Ricorrenza

MergeSort-like: $T(n) = 2T(n/2) + \Theta(n) \rightarrow \Theta(n \log n)$ **QuickSort-like:** $T(n) = T(k) + T(n-k-1) + \Theta(n) \rightarrow$ dipende da k **Ricerca Binaria:** $T(n) = T(n/2) + \Theta(1) \rightarrow \Theta(\log n)$ **Tree Walk:** $T(n) = 2T(n/2) + \Theta(1) \rightarrow \Theta(n)$

Verificare Master Theorem

1. Calcola $\log_b a$
2. Confronta $f(n)$ con $n^{(\log_b a)}$
3. Se differenza polinomiale $\rightarrow$ applica caso 1 o 3
4. Se uguali (a meno di log) $\rightarrow$ caso 2
5. **Caso 3:** verifica sempre la condizione di regolarità!

Induzione Matematica

Struttura standard:

1. **Caso base:** $T(1) = c_1$
2. **Ipotesi induttiva:** Assumi $T(k) \leq f(k)$ per $k < n$
3. **Passo induttivo:** Dimostra $T(n) \leq f(n)$ usando l'ipotesi
4. **Trova costanti:** Determina c, n_0 appropriati